

SOLID Principles - Clean OOP Design Notes

SOLID is a set of five principles that help you write clean, maintainable, and scalable object-oriented code.

- 1 S - Single Responsibility Principle
- 2 O - Open/Closed Principle
- 3 L - Liskov Substitution Principle
- 4 I - Interface Segregation Principle
- 5 D - Dependency Inversion Principle

1) Single Responsibility Principle (SRP)

A class should have only one reason to change (one job).

Wrong:

```
class Report
{
    public void Generate() { }
    public void SaveToFile() { }
    public void SendEmail() { }
}
```

Correct:

```
class Report
{
    public void Generate() { }
}

class ReportSaver
{
    public void SaveToFile() { }
}

class EmailSender
{
    public void SendEmail() { }
}
```

2) Open/Closed Principle (OCP)

Open for extension, closed for modification.

Wrong:

```
class Discount
{
    public double GetDiscount(string type)
    {
        if (type == "Student") return 0.1;
        else if (type == "VIP") return 0.2;
        return 0;
    }
}
```

Correct:

```
interface IDiscount
```

```

{
    double GetDiscount();
}

class StudentDiscount : IDiscount
{
    public double GetDiscount() => 0.1;
}

class VIPDiscount : IDiscount
{
    public double GetDiscount() => 0.2;
}

```

3) Liskov Substitution Principle (LSP)

Derived classes must be replaceable with base classes without breaking behavior.

Wrong:

```

class Bird
{
    public virtual void Fly() { }
}

class Penguin : Bird
{
    public override void Fly()
    {
        throw new Exception("I can't fly!");
    }
}

```

Correct:

```

class Bird { }

class FlyingBird : Bird
{
    public void Fly() { }
}

class Penguin : Bird { }

```

4) Interface Segregation Principle (ISP)

Do not force classes to implement methods they do not use.

Wrong:

```

interface IWorker
{
    void Work();
    void Eat();
}

class Robot : IWorker
{
    public void Work() { }
    public void Eat() { }
}

```

Correct:

```

interface IWorkable
{
    void Work();
}

```

```

    }

    interface IEatable
    {
        void Eat();
    }

    class Robot : IWorkable
    {
        public void Work() { }
    }

```

5) Dependency Inversion Principle (DIP)

Depend on abstractions, not concrete classes.

Wrong:

```

class MySQLDatabase
{
    public void Save() { }
}

class UserService
{
    private MySQLDatabase db = new MySQLDatabase();

    public void SaveUser()
    {
        db.Save();
    }
}

```

Correct:

```

interface IDatabase
{
    void Save();
}

class MySQLDatabase : IDatabase
{
    public void Save() { }
}

class UserService
{
    private IDatabase db;

    public UserService(IDatabase database)
    {
        db = database;
    }

    public void SaveUser()
    {
        db.Save();
    }
}

```

Quick Summary

- 1 SRP: One class = One responsibility
- 2 OCP: Extend without modifying existing code
- 3 LSP: Subclasses should not break expected behavior

- 4 ISP: Use small, specific interfaces
- 5 DIP: Depend on abstractions